Formulación y Búsqueda Ciega en Espacios de Estados: El Problema de los Misioneros y Caníbales

Christopher Pulupa

Facultad de Ingeniería de Sistemas, Escuela Politécnica Nacional,
Quito, Ecuador
Tarea Individual DG_S4 — Inteligencia Artificial

Resumen Se formaliza el problema de los misioneros y caníbales como un problema de búsqueda en espacio de estados (S, s_0, A, T, G) y se implementan en Python las búsquedas no informadas en anchura (BFS, frontera FIFO) y en profundidad (DFS, frontera LIFO), junto con A* como referencia informada. En la instancia clásica ($N = 3$, bote de capacidad $K = 2$) ambos algoritmos ciegos encuentran un camino de 11 cruces expandiendo 14 (BFS) y 11 (DFS) nodos en unas decenas de microsegundos. El análisis empírico muestra que este resultado no contradice la teoría sino que la confirma: la restricción de seguridad reduce el espacio a 16 estados alcanzables con un factor de ramificación efectivo $b^* \approx 1{,}04$, y en ese grafo todos los caminos simples a la meta miden exactamente 11. Al generalizar a N y K mayores, DFS deja de ser óptimo (llega a duplicar la longitud óptima) y, al retirar la detección de estados repetidos, la frontera de BFS crece a 13 835 nodos frente a 5 de DFS, reproduciendo la brecha $O(b^d)$ frente a $O(bm)$.

Keywords: Búsqueda no informada · BFS · DFS · Espacio de estados · Factor de ramificación efectivo · Misioneros y caníbales.

1. Introducción

El problema de los misioneros y caníbales es uno de los ejemplos canónicos de la Inteligencia Artificial para estudiar la *formulación* de problemas: Amarel [1] lo usó para mostrar que la elección de la representación determina la dificultad de la búsqueda. Tres misioneros y tres caníbales deben cruzar un río en un bote de dos plazas sin que, en ninguna orilla, los caníbales superen en número a los misioneros presentes.

Siguiendo el marco de agentes de resolución de problemas de Russell y Norvig [6], el agente (i) formula el objetivo y el problema, (ii) busca una secuencia de acciones en el modelo y (iii) la ejecuta. Este trabajo cubre las dos primeras fases: la Sección 2 define el problema con rigor matemático, la Sección 3 describe su abstracción a código, la Sección 4 presenta el *benchmarking* y la Sección 5 contrasta los resultados con la teoría asintótica.

2. Formulación teórica

Se trabaja con la generalización de N misioneros, N caníbales y un bote de capacidad K; la instancia pedida es $N = 3$, $K = 2$. Un problema de búsqueda es la tupla $\mathcal{P} = (S, s_0, A, \textsc{Aplicable}, T, G, c)$.

Estados. Basta con describir la orilla izquierda, porque la derecha se deduce por complemento. Un estado es la tupla

$$s = (m, c, b), \qquad m, c \in \{0, \dots, N\},\ b \in \{0, 1\}, \tag{1}$$

donde m y c son los misioneros y caníbales en la orilla izquierda y $b = 1$ si el bote está en ella. Las cantidades en la orilla derecha son $\bar{m} = N - m$ y $\bar{c} = N - c$. El conjunto de estados *válidos* es

$$S = \big\{(m, c, b)\ :\ (m = 0 \vee m \geq c)\ \wedge\ (\bar{m} = 0 \vee \bar{m} \geq \bar{c})\big\}. \tag{2}$$

Los pares (m, c) que cumplen (2) son los de $m = 0$, los de $m = N$ y los de la diagonal $m = c$, es decir $3N+1$ pares; por tanto $|S| = 2(3N+1)$, que para $N = 3$ da $|S| = 20$ frente a las $4 \cdot 4 \cdot 2 = 32$ combinaciones sin restricción. Cuatro de ellos, $(0, 0, 1)$, $(3, 3, 0)$, $(0, 3, 0)$ y $(3, 0, 1)$, son inalcanzables desde s_0 (el bote no puede estar en una orilla vacía de la que nadie lo trajo), de modo que el espacio alcanzable tiene $|S_r| = 16$ estados.

Estado inicial y estado objetivo.

$$s_0 = (N, N, 1) = (3, 3, 1), \qquad G = \{(0, 0, 0)\}. \tag{3}$$

Acciones. Una acción es la carga del bote, $a = (\delta_m, \delta_c)$, con

$$A = \{(\delta_m, \delta_c) \in \mathbb{N}^2 : 1 \leq \delta_m + \delta_c \leq K\}, \qquad |A| = \tfrac{K(K+3)}{2}. \tag{4}$$

Para $K = 2$, $A = \{(2, 0), (1, 1), (0, 2), (1, 0), (0, 1)\}$. La acción es *aplicable* en s si las personas están en la orilla donde se encuentra el bote:

$$\textsc{Aplicable}(s, a) \iff \begin{cases} \delta_m \leq m\ \wedge\ \delta_c \leq c, & b = 1, \\ \delta_m \leq N - m\ \wedge\ \delta_c \leq N - c, & b = 0. \end{cases} \tag{5}$$

Modelo de transición. El bote cambia de orilla y lleva consigo la carga:

$$T\big((m, c, b), (\delta_m, \delta_c)\big) = \begin{cases} (m - \delta_m,\ c - \delta_c,\ 0), & b = 1\ \text{(ida)}, \\ (m + \delta_m,\ c + \delta_c,\ 1), & b = 0\ \text{(vuelta)}, \end{cases} \tag{6}$$

y la función de sucesores conserva solo los resultados seguros:

$$\textsc{Suc}(s) = \{(a, T(s, a)) : \textsc{Aplicable}(s, a) \wedge T(s, a) \in S\}. \tag{7}$$

Toda transición es reversible, por lo que el grafo de estados es no dirigido.

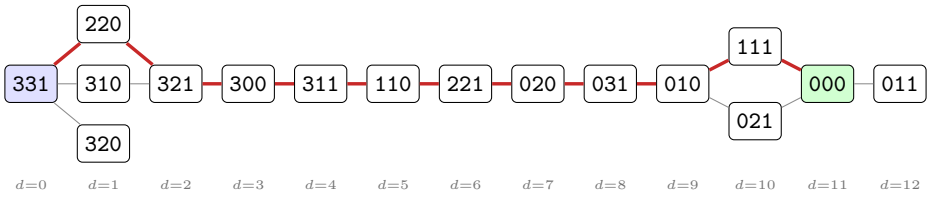


Figura 1. Grafo del espacio de estados alcanzable ($|S_r| = 16$, 17 aristas) para $N = 3$, $K = 2$. La etiqueta `mcb` denota el estado (m, c, b) y la columna indica la profundidad mínima desde s_0. En rojo, el camino de 11 cruces encontrado por BFS. Las únicas bifurcaciones son la primera y la última jugada; $(3, 2, 0)$ y $(0, 1, 1)$ son callejones sin salida.

Test objetivo y costo. OBJETIVO$(s) \iff s = (0, 0, 0)$. Cada cruce cuesta $c(s, a, s') = 1$, así que el costo de un camino es su número de acciones y el camino óptimo es el más corto.

La Fig. 1 muestra el grafo completo de S_r para $N = 3$, $K = 2$, ordenado por profundidad desde s_0. Tiene 17 aristas y un grado medio de 2,125, muy por debajo del máximo teórico $|A| = 5$: la restricción (2) poda la mayor parte de las acciones.

3. Implementación

El código está en Python 3 sin dependencias externas y se organiza como un agente de resolución de problemas con responsabilidades separadas: el módulo `src/problema.py` encapsula la formulación $\mathcal{P}$, con los estados como tuplas inmutables `(m_izq, c_izq, bote_izq)` que sirven de clave en tablas hash; el módulo `src/busqueda.py` contiene algoritmos que solo conocen la interfaz del problema (`estado_inicial`, `sucesores` y `es_objetivo`); y el script `benchmark.py` ejecuta los experimentos. El repositorio incluye pruebas unitarias que verifican la formulación, la validez de cada camino y la consistencia de la heurística.

Poda en la generación. La restricción (2) se aplica dentro de SUC (Listado 1): un estado inseguro nunca se crea ni entra a la frontera. Es la forma más simple de propagación de restricciones y evita gastar expansiones en estados que el test objetivo rechazaría más tarde.

```
def sucesores(self, s):
    for a in self.acciones: # (dm, dc), 1 <= dm + dc <= K
        if self.es_aplicable(s, a):
            s2 = self.resultado(s, a) # modelo de transición T(s, a)
            if self.es_valido(s2): # restricción de seguridad
                yield a, s2
```

Listado 1. Función de sucesores con poda de estados inseguros.

BFS y DFS. Ambos son búsquedas de *grafo*: guardan los estados ya vistos para no expandir dos veces el mismo estado. BFS usa `collections.deque` como cola FIFO y marca un estado como alcanzado al generarlo; DFS usa una lista como pila LIFO y marca al expandir, apilando los hijos en orden inverso para respetar el orden de las acciones (Listado 2). Sin el conjunto de explorados, DFS no termina: la acción $(0, 1)$ seguida de su inversa genera un ciclo infinito. En ambos el test objetivo se aplica al extraer el nodo, con lo que los contadores de nodos expandidos son directamente comparables.

```
frontera, alcanzados = deque([Nodo(s0)]), {s0}
while frontera:
    nodo = frontera.popleft() # FIFO
    if problema.es_objetivo(nodo.estado):
        return nodo # el camino se reconstruye por los padres
    expandidos += 1
    for accion, s2 in problema.sucesores(nodo.estado):
        if s2 not in alcanzados:
            alcanzados.add(s2)
            frontera.append(Nodo(s2, nodo, accion, nodo.profundidad + 1))
```

Listado 2. Núcleo de BFS; DFS cambia `popleft()` por `pop()`.

Variantes de árbol y A*. Para aislar el efecto de la estrategia sobre la memoria se implementaron dos variantes sin conjunto global de explorados: BFS de árbol, y DFS de árbol que solo evita repetir estados del camino actual, con memoria $O(bm)$. Además se incluyó A* [3] con una heurística obtenida por *relajación* [5]: se ignora la restricción (2) y se cuenta el mínimo de cruces para pasar $n = m + c$ personas. Cada viaje de ida y vuelta traslada netamente $K - 1$ personas y el último traslada K; si el bote está a la derecha, primero debe volver con al menos una persona. De ahí

$$h_1(n) = \begin{cases} 0, & n = 0 \\ 1, & 0 < n \le K \\ 2\left\lceil \frac{n-K}{K-1} \right\rceil + 1, & n > K \end{cases} \tag{8}$$

$$h(m, c, b) = \begin{cases} h_1(n), & b = 1 \\ 1 + h_1(n+1), & b = 0,\ n > 0 \\ 0, & b = 0,\ n = 0 \end{cases} \tag{9}$$

Al ser el costo exacto de un problema relajado, h es admisible y consistente; las pruebas lo comprueban sobre todas las aristas ($h(s) \le 1 + h(s')$, $h(s_G) = 0$). Para s_0 con $K = 2$ se obtiene $h(s_0) = 9$, frente al costo real $h^*(s_0) = 11$.

4. Benchmarking

Protocolo. Las pruebas se ejecutaron con Python 3.13.18 en Linux x86_64. El tiempo reportado es la *mediana* de 2 000 ejecuciones (10 para BFS de árbol)

Tabla 1. Comparación BFS frente a DFS en la instancia clásica ($N = 3$, $K = 2$).

Algoritmo	Nodos expandidos	Tiempo de ejecución (s)	Longitud del camino
BFS (cola)	14	2.65×10^{-5}	11
DFS (pila)	11	2.46×10^{-5}	11

Tabla 2. Métricas completas para $N = 3$, $K = 2$. Exp.: nodos expandidos; Gen.: nodos generados; $|F|_{\text{máx}}$: tamaño máximo de la frontera; Mem.: máximo de nodos simultáneos y pico de memoria (`tracemalloc`); d: longitud del camino. Debajo de la línea, variantes sin detección de estados repetidos.

Algoritmo	Exp.	Gen.	$\lvert F\rvert_{\text{máx}}$	Mem. (nodos)	Mem. (KiB)	Tiempo (ms)	d	b^*
BFS	14	14	3	15	3.2	0.0265	11	1.040
DFS	11	15	5	16	2.9	0.0246	11	1.051
A*	14	14	3	15	3.4	0.0342	11	1.040
BFS (árbol)	10 444	24 278	13 835	13 835	3 971.5	23.3096	11	2.383
DFS (árbol)	11	15	5	5	5.1	0.0260	11	1.051

medidas con el reloj de alta resolución `perf_counter` de Python, lo que amortigua el ruido del sistema operativo en tiempos de microsegundos. La memoria se reporta de dos formas: en nodos, como el máximo de $|F| + |E|$ (frontera más explorados), y en bytes, como el pico medido con `tracemalloc` en una ejecución separada para no sesgar el tiempo. El factor de ramificación efectivo b^* se obtiene resolviendo por bisección $N_{\text{gen}} + 1 = \sum_{i=0}^{d} (b^*)^i$, donde N_{gen} son los nodos generados y d la profundidad de la solución [6]. Todo se reproduce con `python3 benchmark.py`.

Resultados de la instancia clásica. La Tabla 1 es la comparación que pide el enunciado y la Tabla 2 la amplía con las demás métricas.

Los dos algoritmos ciegos encuentran un camino de longitud $d = 11$, que es el óptimo conocido. DFS expande menos nodos (11 frente a 14) porque su primera rama llega directamente a la meta, mientras que BFS completa cada nivel, incluido el callejón $(3, 2, 0)$ y los estados del nivel 10. Las diferencias de tiempo (26,5 frente a 24,6 μs) son del orden del ruido de medición y no permiten afirmar que uno sea más rápido. $b^* \approx 1{,}04$ indica que la búsqueda es casi lineal. El contraste aparece en las variantes de árbol: sin detectar repetidos, BFS expande 10 444 nodos, mantiene 13 835 en la frontera, usa 3,9 MiB y tarda 23 ms (unas 880 veces más que BFS de grafo), con un b^* aparente de 2,38; DFS de árbol, con control de ciclos en el camino, conserva 5 nodos.

Sensibilidad de DFS al orden de las acciones. El camino que devuelve DFS depende del orden en que se prueban las acciones. Se ejecutó DFS con todas las permutaciones de A (Tabla 3). En la instancia clásica las 120 permutaciones

Tabla 3. Longitud del camino de DFS sobre todas las permutaciones de las acciones.

				Órdenes con DFS de longitud			
N	K	Permutaciones	d^* (BFS)	$d = 11$	$d = 13$	$d = 15$	% óptimo
3	2	120	11	120	0	0	100,0
5	3	362 880	11	59 832	43 632	259 416	16,5

Tabla 4. Escalabilidad de las búsquedas de grafo. $|S_r|$: estados alcanzables; Exp.: nodos expandidos; d: longitud del camino; ms: mediana del tiempo.

			BFS			DFS			A*		
N	K	$\lvert S_r\rvert$	Exp.	d	ms	Exp.	d	ms	Exp.	d	ms
3	2	16	14	11	0.026	11	11	0.024	14	11	0.035
5	3	28	26	11	0.072	16	15	0.054	23	11	0.079
10	4	58	56	17	0.223	36	29	0.172	48	17	0.225
25	4	148	118	47	0.498	96	59	0.459	108	47	0.528
50	5	298	223	93	1.282	193	111	1.240	209	93	1.356
100	5	598	423	193	2.499	393	211	2.561	409	193	2.727
200	6	1198	825	199	6.419	795	415	6.836	814	199	7.046
500	6	2998	2025	499	17.081	1995	1015	18.840	2014	499	19.201

dan $d = 11$; con $N = 5$, $K = 3$ ($|A| = 9$, $9! = 362\,880$ órdenes) solo el 16,5 % encuentra el óptimo.

Escalabilidad. La Tabla 4 y la Fig. 2 varían N y K (con $K = 2$ no existe solución para $N \geq 4$, lo que ambos algoritmos detectan al agotar el espacio). $|S_r|$ crece linealmente ($6N - 2$ para estas instancias) y los nodos expandidos por los tres algoritmos lo siguen. A medida que el problema crece, DFS expande algo menos que BFS, pero su camino se aleja del óptimo hasta duplicarlo ($d = 1\,015$ frente a 499 con $N = 500$). La frontera de BFS se mantiene en $|F|_{\text{máx}} \leq 14$, mientras que la de DFS crece con N hasta 2 520 nodos. A* coincide siempre con el óptimo de BFS, pero ahorra pocas expansiones (2 014 frente a 2 025 con $N = 500$).

5. Discusión y conclusión crítica

¿Por qué difieren los resultados asintóticos? La teoría [6] da para BFS tiempo y espacio $O(b^d)$ ($O(b^{d+1})$ con el test objetivo al expandir, como aquí), y para DFS tiempo $O(b^m)$ y espacio $O(bm)$, donde b es el factor de ramificación, d la profundidad de la solución más superficial y m la profundidad máxima. Estas cotas suponen búsqueda en árbol sobre un espacio grande. En este problema hay dos hechos que cambian el comportamiento observado:

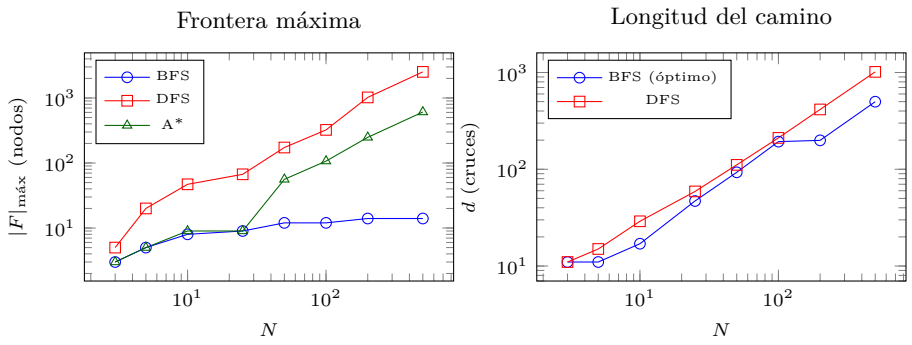


Figura 2. Frontera máxima y longitud del camino en función de N (instancias de la Tabla 4, ejes logarítmicos).

Tabla 5. Memoria de BFS y DFS (búsqueda de grafo) por instancia.

		BFS		DFS	
N	K	$\lvert F\rvert_{\text{máx}}$	Mem. (KiB)	$\lvert F\rvert_{\text{máx}}$	Mem. (KiB)
3	2	3	3.2	5	2.9
5	3	5	5.2	20	4.9
10	4	8	6.8	47	10.9
25	4	9	16.6	67	24.5
50	5	12	22.1	173	45.7
100	5	12	56.4	323	116.3
200	6	14	89.4	1020	265.4
500	6	14	350.3	2520	983.7

1. *El espacio es pequeño y está fuertemente podado.* Con detección de repetidos, ambos algoritmos quedan acotados por $O(|S_r| + |E|)$ en tiempo y $O(|S_r|)$ en memoria [2], con $|S_r| = 16$ para $N = 3$ y lineal en N en general. Por eso los dos expanden casi todo el espacio, y sus tiempos son comparables y crecen de forma polinómica moderada con N (Tabla 4), lejos del crecimiento exponencial de la cota de árbol. El factor $b^* \approx 1{,}04$ significa que los nodos generados (14–15) apenas superan la profundidad de la solución ($d = 11$): la búsqueda es casi lineal.
2. *La estructura del grafo es casi una cadena.* La Fig. 1 muestra que solo hay bifurcaciones al principio y al final. La profundidad máxima m coincide prácticamente con d, así que la ventaja teórica de DFS en memoria (bm frente a b^d) y su riesgo de caminos largos ($m \gg d$) no tienen espacio para manifestarse.

Cuando estas condiciones desaparecen, las diferencias asintóticas vuelven: sin detección de repetidos el espacio efectivo pasa a ser el árbol de búsqueda, y BFS expande 10 444 nodos con $b^* = 2{,}38$, mientras que al ampliar N y K el grafo

deja de ser una cadena y DFS empieza a devolver caminos cada vez más largos que el óptimo.

¿DFS garantizó el camino más corto? No. En la instancia clásica DFS devolvió $d = 11$ con cualquier orden de las acciones, pero porque *todos los caminos simples* de s_0 a la meta miden exactamente 11 (verificado por enumeración exhaustiva). Fue una consecuencia de la estructura del problema, no una propiedad del algoritmo. DFS no es óptimo: devuelve el primer camino que encuentra su rama más profunda. Con $N = 5$, $K = 3$, el 83,5 % de los órdenes produce caminos de 13 o 15 cruces, y con $N = 500$ el camino de DFS es 2,03 veces el óptimo. BFS, en cambio, es óptimo porque los costos son unitarios: explora por niveles, así que la primera meta extraída está a profundidad mínima.

¿Por qué BFS consumió más memoria? BFS debe mantener toda la frontera de un nivel antes de pasar al siguiente, y el tamaño de un nivel crece como b^d; además, para reconstruir caminos, cada nodo guarda su padre. DFS solo necesita la rama actual y los hermanos pendientes, $O(bm)$. Los datos lo confirman en el régimen donde la teoría aplica: en búsqueda de árbol BFS llega a 13 835 nodos en la frontera (≈3,9 MiB) frente a 5 nodos de DFS, una razón de 2 767:1. Es importante, sin embargo, ser precisos con la búsqueda de grafo de este problema: allí la memoria la domina el conjunto de explorados, de tamaño $|S_r|$ en ambos casos, y la diferencia es mínima (15 frente a 16 nodos; 3,2 frente a 2,9 KiB). Incluso, al crecer N, la frontera de DFS supera a la de BFS (Tabla 5), porque la pila conserva hermanos y duplicados que aún no ha descartado, mientras que el ancho de cada nivel de BFS está acotado por la propia restricción de seguridad. La afirmación *"BFS consume más memoria"* es correcta como cota del peor caso ($O(b^d)$ frente a $O(bm)$), pero su magnitud real depende del factor de ramificación efectivo y de si se eliminan los estados repetidos.

Conclusión. La formulación (m, c, b) con poda en la función de sucesores reduce el problema a 16 estados alcanzables, de modo que BFS y DFS lo resuelven en microsegundos. Para este tipo de problema, pequeño, con costos unitarios y donde se requiere el camino mínimo, BFS con detección de repetidos es la elección correcta: garantiza optimalidad y su costo en memoria queda acotado por $|S_r|$. DFS solo es preferible si la memoria es el recurso crítico y no se exige optimalidad. En ese caso, la profundización iterativa [4] combina la memoria $O(bd)$ de DFS con la optimalidad de BFS. La heurística relajada de A* ahorra pocas expansiones porque ignora justo la restricción que hace difícil el problema. Una heurística más informada debería incorporar la condición de seguridad.

Código fuente:
`https://github.com/USUARIO/misioneros-canibales-busqueda`

Referencias

1. Amarel, S.: On representations of problems of reasoning about actions. In: Michie, D. (ed.) Machine Intelligence 3, pp. 131–171. Edinburgh University Press (1968)
2. Cormen, T.H., Leiserson, C.E., Rivest, R.L., Stein, C.: Introduction to Algorithms. MIT Press, Cambridge, MA, 4 edn. (2022)
3. Hart, P.E., Nilsson, N.J., Raphael, B.: A formal basis for the heuristic determination of minimum cost paths. IEEE Transactions on Systems Science and Cybernetics **4**(2), 100–107 (1968)
4. Korf, R.E.: Depth-first iterative-deepening: An optimal admissible tree search. Artificial Intelligence **27**(1), 97–109 (1985)
5. Pearl, J.: Heuristics: Intelligent Search Strategies for Computer Problem Solving. Addison-Wesley, Reading, MA (1984)
6. Russell, S., Norvig, P.: Artificial Intelligence: A Modern Approach. Pearson, Hoboken, NJ, 4 edn. (2020)